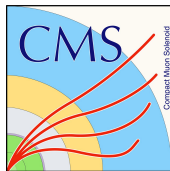


Advanced Support for Heterogeneous Resources for the Future CMS Offline Computing

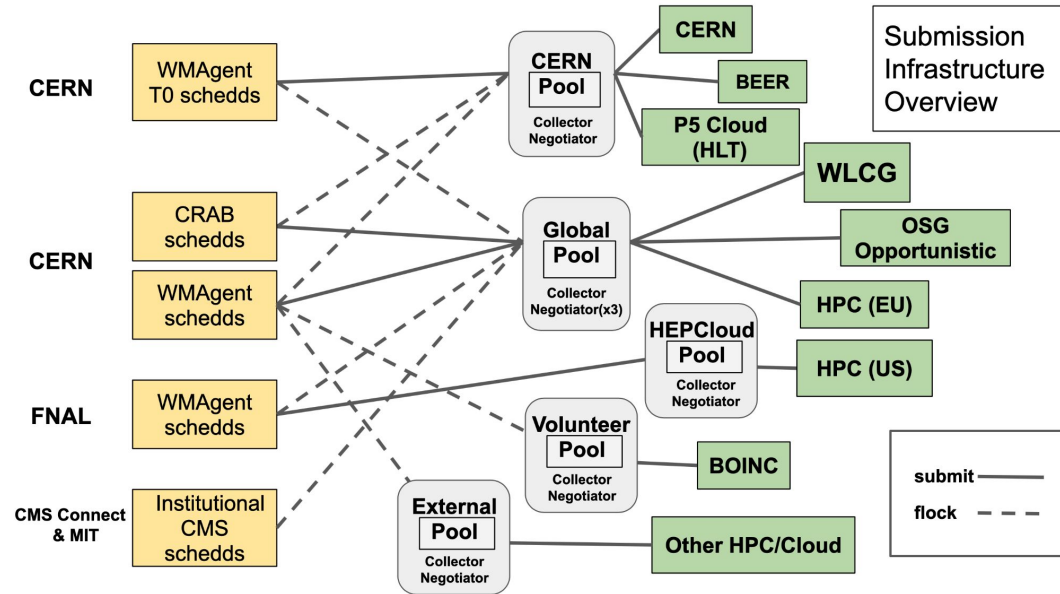
R. Florian von Cube, A. Pérez-Calero Yzquierdo, Marco Mascheroni,
Hyun Woo Kim, Vaiva Zokaitė, Luís Simas
for the CMS Collaboration

CHEP 2026



CMS Submission Infrastructure

- CMS Submission Infrastructure (SI) operates CMS' computing resource pools
- Nowadays **on average 500k CPU cores**, globally distributed over ~160 WLCG compute centers
- Already today **including many HPC centers** in Europe and the US



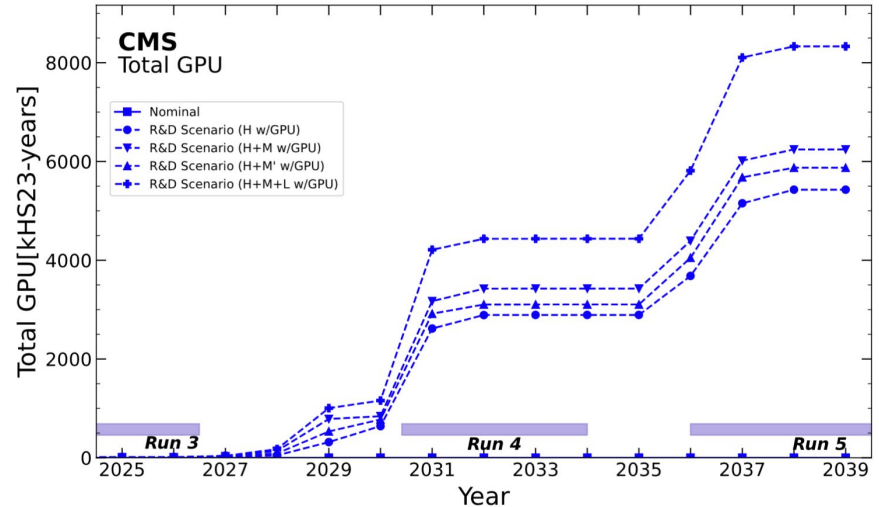
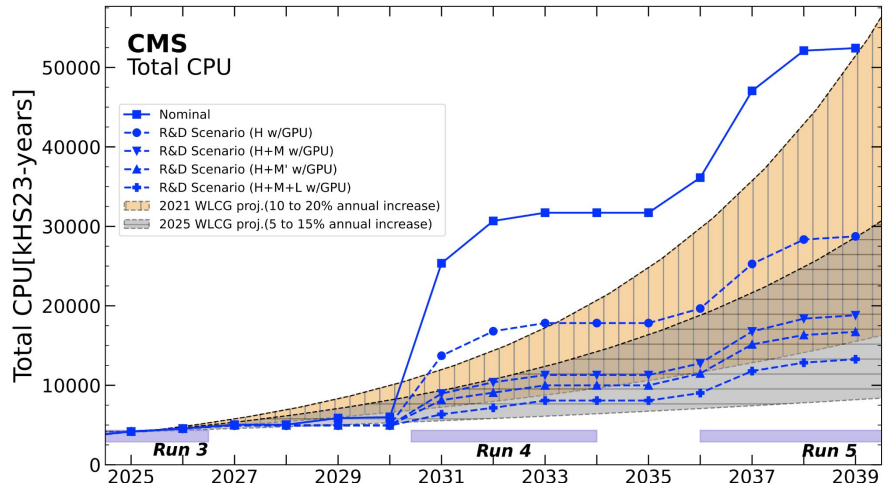
The HL-LHC Challenge

- Significant increase of demand for resources expected for Run 4 and 5, conventional CPUs as well as GPUs
- Challenge for SI: enable further scalability while maintaining **stability**, **flexibility**, and scheduling **efficiency**
- Profit from current system, expand and adapt to **new resource types** and tools

See [CMS Offline Software and Computing for HL-LHC Conceptual Design Report](#)



Heterogeneous Resources for Future CMS Offline Computing
R. Florian von Cube for the CMS Collaboration



Heterogeneous Resources

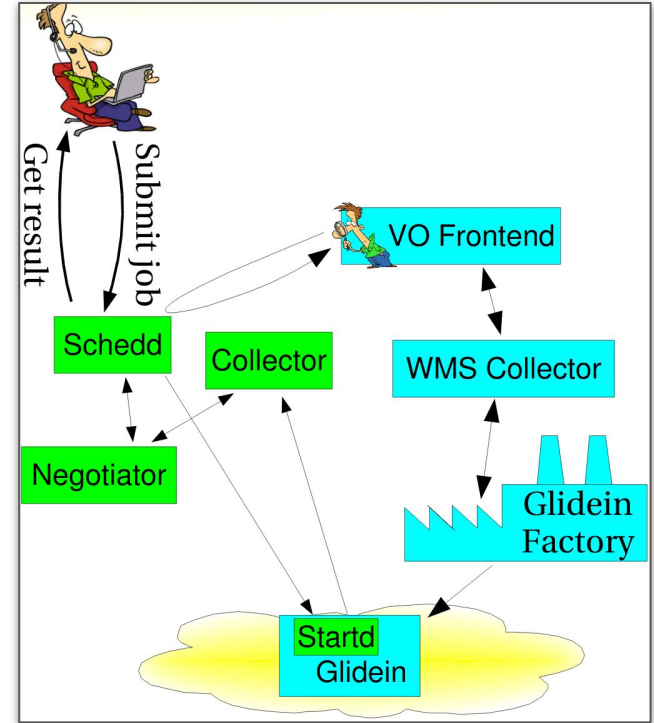
- General trend in HEP towards large-scale shared facilities with other communities
- Need for non-x86 architectures for running analysis tools
 - ARM
 - GPUs (NVIDIA, AMD)
- Main areas for development in CMS SI towards the HL-LHC:
 - Integration of new resources
 - Proof of readiness for scaling of the system
 - Effective use of heterogeneous resources



Resource Acquisition and Matchmaking

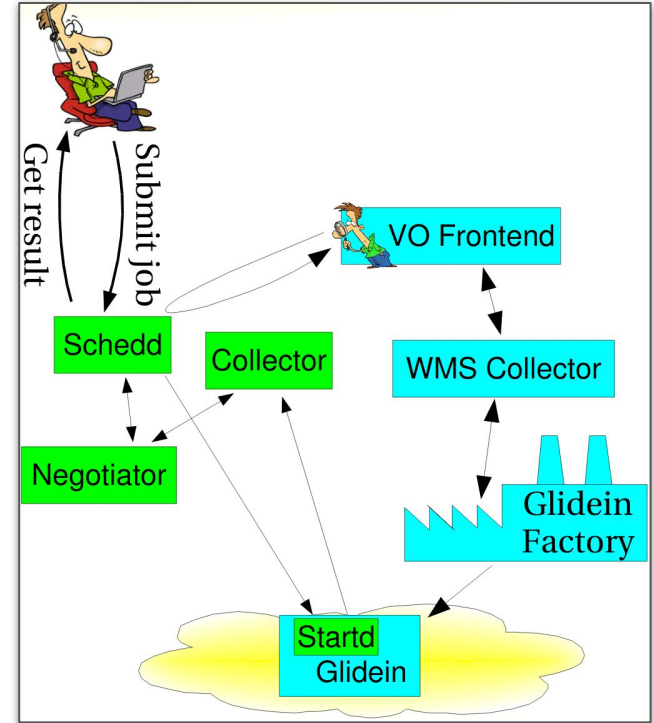
After submission of payload-jobs, the matchmaking happens in two steps:

1. **GlideinWMS submits Glideins** (resource requests) **to sites**. The Glidein starts the **StartD** **which joins the HTCondor resource pool**
2. Once the StartD is connected to the pool, job-resource negotiation starts, and payload-jobs are executed



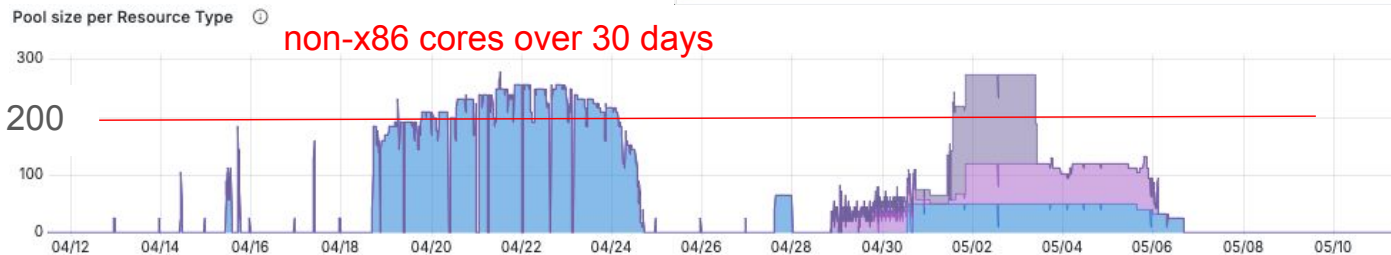
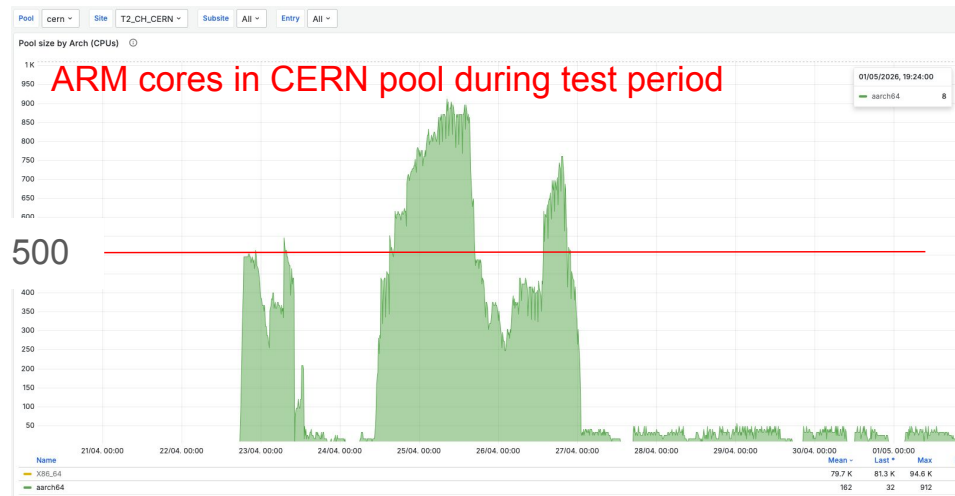
Scheduling ARM Resources

- **Factory** has entries configured for ARM resources
- **VO Frontend** selects entries compatible with DESIRED_OS and architecture.
 - Architecture can be “x86_64”, “aarch64”, or “any”
 - Payloads are started in container images specific to the architecture and OS
- HTCondor binaries are shipped with Glidein for target architecture
- **HTCondor matchmaking** works as usual
 - ARM slots are standard resources once requirements are matched
- Jobs can specify REQUIRED_ARCH as a list of compatible architectures, or any



ARM resources in resource pools

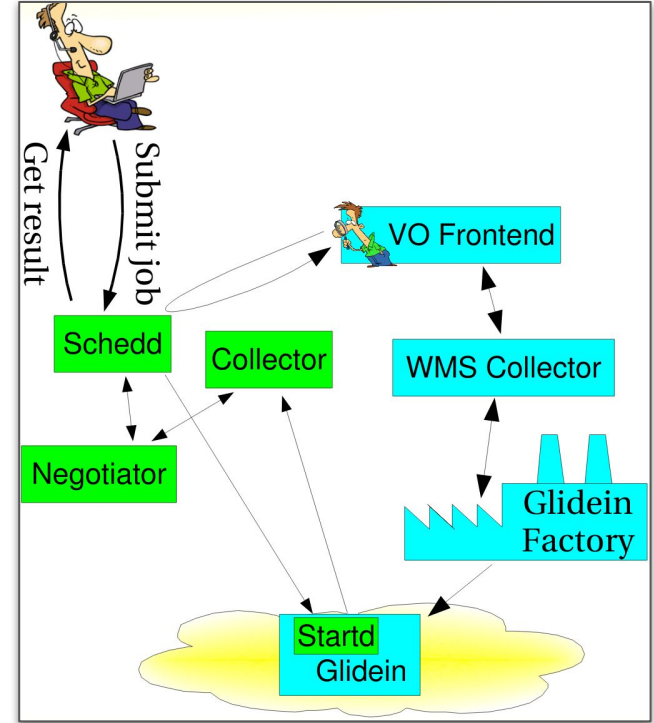
- The presented setup allows for the integration of ARM resources
- Overall fraction of ARM resources still very low and only at a few sites
- CMS SI ready for allocation of ARM resources



Scheduling GPU Resources

Integration of GPU resources similar to ARM resources:

- **Glidein Factory** has entries configured for GPU resources
- **VO Frontend** selects entries providing GPU resources
- **HTCondor matchmaking** works as usual, taking GPU requirements into account

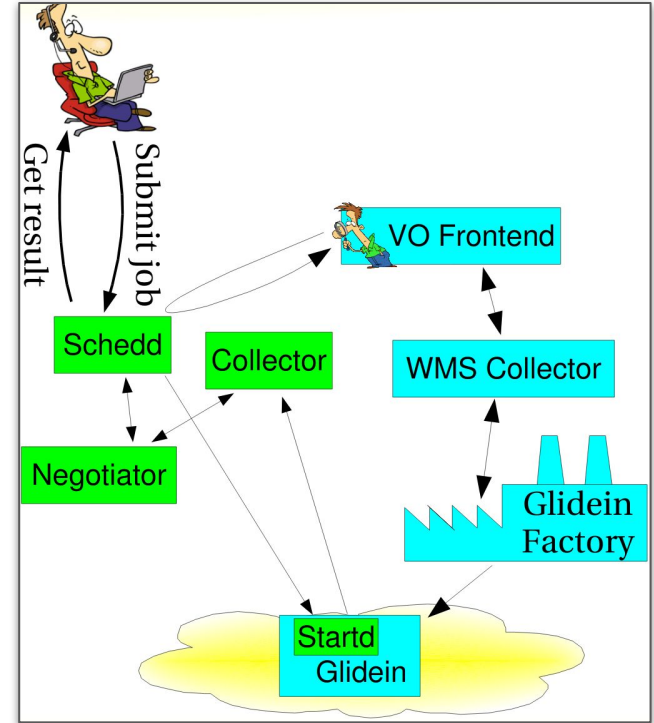


GPU Requests in CMS Workflow Management

- The CMS Workflow Management (WM) submits jobs to SI resource pools
- A common set of attributes for requesting GPUs in the usual format is defined:
 - RequiresGPU
 - RequestGPUs
 - DESIRED_GPUMemoryMB
 - DESIRED_GPUMinimumCapability
 - DESIRED_GPURuntime
- Any additional requirement is also possible:
 - `JobExtraMatchRequirements = 'regexp("^NVIDIA L40S$", GPUs_DeviceName)'`

Triggering GPU Glideins

- RequiresGPU and RequestGPUs control the matchmaking:
 - RequiresGPU triggers GPU-Glideins in first matchmaking
 - RequestGPUs used for slot matching in HTCondor
- These can cover three use cases:
 - Job needs GPU resources
(RequiresGPU = True && RequestGPUs > 0)
 - Job can use GPU resources
(RequiresGPU = False && RequestGPUs > 0)
 - Job uses only CPUs
(RequiresGPU = False && RequestGPUs = 0)



Discovery of GPUs: condor_gpu_discovery

- HTCondor runs condor_gpu_discovery on startd startup
 - Detects devices using NVidia driver, HIP 6, and OpenCL libraries
 - Attributes are advertised in slot classads
 - CMS SI does not yet have experience with non-NVidia GPUs
- Allows for fine-grained match-making based on device properties

```
GPUs = 1
GPUs_Capability = 8.0
GPUs_ClockMhz = 1410.0
GPUs_ComputeUnits = 108
GPUs_CoresPerCU = 64
GPUs_DeviceName = "NVIDIA A100-SXM4-40GB"
GPUs_DevicePciBusId = "0000:CA:00.0"
GPUs_DeviceUuid =
    "ec8f9563-d25f-b6ad-7c25-bd8cd1487766"
GPUs_DriverVersion = 12.8
GPUs_ECCEnabled = true
GPUs_GlobalMemoryMb = 40442
GPUs_GPU_ec8f9563 = [ Id =
    "GPU-ec8f9563"; ClockMhz = 1410.0;
    Capability = 8.0; CoresPerCU = 64;
    DeviceName = "NVIDIA A100-SXM4-40GB";
    DeviceUuid =
    "ec8f9563-d25f-b6ad-7c25-bd8cd1487766";
    ECCEnabled = true; ComputeUnits = 108;
    DriverVersion = 12.8; DevicePciBusId =
    "0000:CA:00.0"; GlobalMemoryMb = 40442;
    MaxSupportedVersion = 12080 ]
GPUs_MaxSupportedVersion = 12080
```

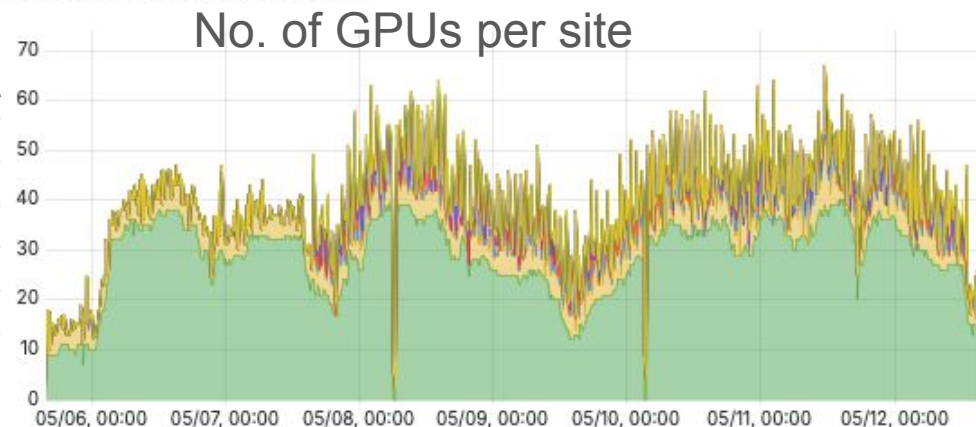
GPU Integration

- SI configures entries for sites providing GPUs
- Glideins use HTCondor's [condor_gpu_discovery](#) to gather device attributes, information is published in the slots attributes.
- SI publishes gathered data in overview dashboard

GPU resources

Site ▼	Host ▼	N_GPUs ▼	CMS_CUDA_SUPI ▼	CMS_NVIDIA_DRI ▼	CPUs ▼
T2_CH_CERN	b9g47n8368.	1	12.0,12.1,12.2,12.3,12.4	580.95.05	8
T1_DE_KIT	hkn0431.loc	1	12.0,12.1,12.2,12.3,12.4		8
T1_DE_KIT	hkn0433.loc	1	12.0,12.1,12.2,12.3,12.4		8
T1_DE_KIT	hkn0702.loc	1	12.0,12.1,12.2,12.3,12.4		8
T1_ES_PIC	gpu15.pic.es	1	12.0,12.1,12.2,12.3,12.4	580.95.05	8

GPU Pool size per resource provider



Overview of GPUs in the Pools

Table with all GPUs in the resource pools, dynamically created

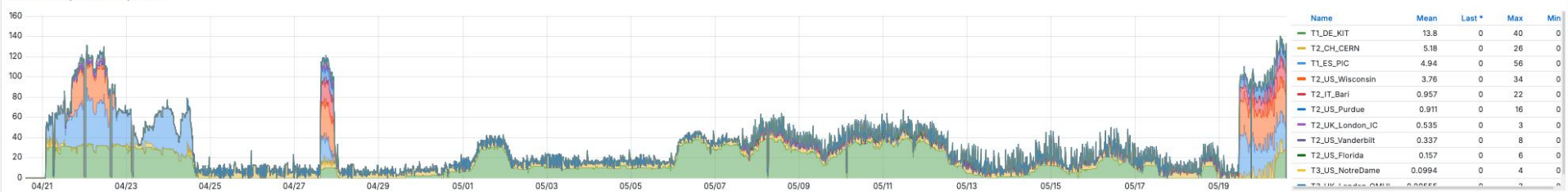
Site	Host	N_GPUs	CMS_CUDA_SUPPORT	CMS_NVIDIA_DRIVE	CPU	TotalMemory	Entry_Name	AssignedGPU	Capability	ClockMhz	ComputeUnits	CoresPerCU	DeviceName	DriverVersion	MaxSupportedVers	ECCEEnabled	GlobalMemoryMb
T2_CH_CERN	b9g57n6415.i	1	12.0,12.1,12.2,12.3,12.4,12.5	580.95.05	8	8000	CMSHTPC_T2_CH_CERN	GPU-bt587b5f,	8.0	1410	108	64	NVIDIA A100-PCIE-40GB	13.0	13000	true	40441
T2_CH_CERN	b9g57n5751.i	1	12.0,12.1,12.2,12.3,12.4,12.5	580.95.05	8	8000	CMSHTPC_T2_CH_CERN	GPU-cdacb063	8.0	1410	108	64	NVIDIA A100-PCIE-40GB	13.0	13000	true	40441
T2_IT_Bari	wn-gpu-8-3-	1	12.0,12.1,12.2,12.3,12.4,12.5	580.105.08	8	20240	CMSHTPC_T2_IT_Bari	GPU-1cf2e1fc,	8.0	1410	108	64	NVIDIA A100-PCIE-40GB	13.0	13000	true	40441
T2_CH_CERN	b9g57n0002.i	1	12.0,12.1,12.2,12.3,12.4,12.5	580.105.08	8	8000	CMSHTPC_T2_CH_CERN	GPU-4ca02207	8.0	1410	108	64	NVIDIA A100-PCIE-40GB	13.0	13000	true	40441
T1_DE_KIT	f03-001-179-	1	10.1,10.2,11.0,11.1,11.2,11.3,	580.95.05	8	20000	CMSHTPC_T1_DE_KIT	GPU-edf948a0	8.0	1410	108	64	NVIDIA A100-PCIE-40GB	13.0	13000	true	40441
T1_DE_KIT	f03-001-175-	1	10.1,10.2,11.0,11.1,11.2,11.3,	580.95.05	8	20000	CMSHTPC_T1_DE_KIT	GPU-f4eedd34,	8.0	1410	108	64	NVIDIA A100-PCIE-40GB	13.0	13000	true	40441
T2_CH_CERN	b9g57n5297.i	1	12.0,12.1,12.2,12.3,12.4,12.5	580.95.05	8	8000	CMSHTPC_T2_CH_CERN	GPU-851bec56	8.0	1410	108	64	NVIDIA A100-PCIE-40GB	13.0	13000	true	40441
T2_CH_CERN	b9g57n0005.i	1	12.0,12.1,12.2,12.3,12.4,12.5	580.105.08	8	8000	CMSHTPC_T2_CH_CERN	GPU-f7b29eb4	8.0	1410	108	64	NVIDIA A100-PCIE-40GB	13.0	13000	true	40441
T2_CH_CERN	b9g57n7738.i	1	12.0,12.1,12.2,12.3,12.4,12.5	580.95.05	8	8000	CMSHTPC_T2_CH_CERN	GPU-7a483b10,	8.0	1410	108	64	NVIDIA A100-PCIE-40GB	13.0	13000	true	40441
T2_CH_CERN	b9g57n5419.i	1	12.0,12.1,12.2,12.3,12.4,12.5	580.95.05	8	8000	CMSHTPC_T2_CH_CERN	GPU-8e36ebf7,	8.0	1410	108	64	NVIDIA A100-PCIE-40GB	13.0	13000	true	40441
T2_CH_CERN	b9g57n7827.i	1	12.0,12.1,12.2,12.3,12.4,12.5	580.95.05	8	8000	CMSHTPC_T2_CH_CERN	GPU-ee78d802	8.0	1410	108	64	NVIDIA A100-PCIE-40GB	13.0	13000	true	40441
T2_CH_CERN	b9g57n0991.i	1	12.0,12.1,12.2,12.3,12.4,12.5	580.95.05	8	8000	CMSHTPC_T2_CH_CERN	GPU-481212f1,	8.0	1410	108	64	NVIDIA A100-PCIE-40GB	13.0	13000	true	40441
T2_CH_CERN	b9g57n2934.i	1	12.0,12.1,12.2,12.3,12.4,12.5	580.95.05	8	8000	CMSHTPC_T2_CH_CERN	GPU-1d17f189,	8.0	1410	108	64	NVIDIA A100-PCIE-40GB	13.0	13000	true	40441
T2_CH_CERN	b9g57n4705.i	1	12.0,12.1,12.2,12.3,12.4,12.5	580.95.05	8	8000	CMSHTPC_T2_CH_CERN	GPU-4cb0c17c,	8.0	1410	108	64	NVIDIA A100-PCIE-40GB	13.0	13000	true	40441
T2_CH_CERN	b9g57n6407.i	1	12.0,12.1,12.2,12.3,12.4,12.5	580.95.05	8	8000	CMSHTPC_T2_CH_CERN	GPU-47e13020	8.0	1410	108	64	NVIDIA A100-PCIE-40GB	13.0	13000	true	40441
T2_CH_CERN	b9g57n1627.c	1	12.0,12.1,12.2,12.3,12.4,12.5	580.105.08	8	8000	CMSHTPC_T2_CH_CERN	GPU-17f035a0,	8.0	1410	108	64	NVIDIA A100-PCIE-40GB	13.0	13000	true	40441
T2_CH_CERN	b9g57n3665.i	1	12.0,12.1,12.2,12.3,12.4,12.5	580.95.05	8	8000	CMSHTPC_T2_CH_CERN	GPU-836178fb,	8.0	1410	108	64	NVIDIA A100-PCIE-40GB	13.0	13000	true	40441
T2_CH_CERN	b9g57n6574.i	1	12.0,12.1,12.2,12.3,12.4,12.5	580.105.08	8	8000	CMSHTPC_T2_CH_CERN	GPU-02931ae6	8.0	1410	108	64	NVIDIA A100-PCIE-40GB	13.0	13000	true	40441
T2_CH_CERN	b9g57n4074.i	1	12.0,12.1,12.2,12.3,12.4,12.5	580.105.08	8	8000	CMSHTPC_T2_CH_CERN	GPU-1e7d9dcf,	8.0	1410	108	64	NVIDIA A100-PCIE-40GB	13.0	13000	true	40441
T2_CH_CERN	b9g57n0001.i	1	12.0,12.1,12.2,12.3,12.4,12.5	580.105.08	8	8000	CMSHTPC_T2_CH_CERN	GPU-668b0fca	8.0	1410	108	64	NVIDIA A100-PCIE-40GB	13.0	13000	true	40441



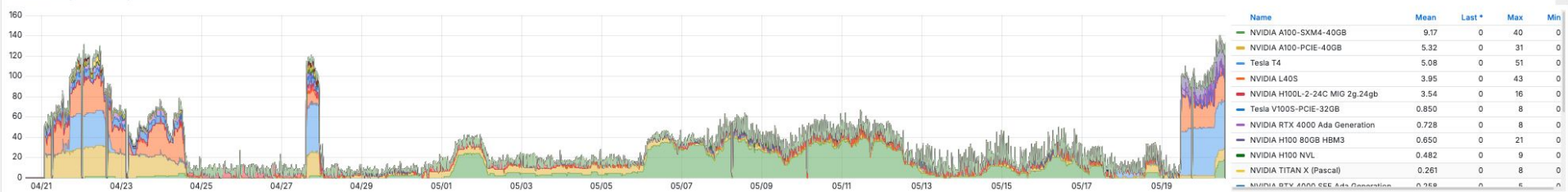
Overview of GPUs in the Pools

Amount of GPUs by site and GPU model

GPU Pool size per resource provider



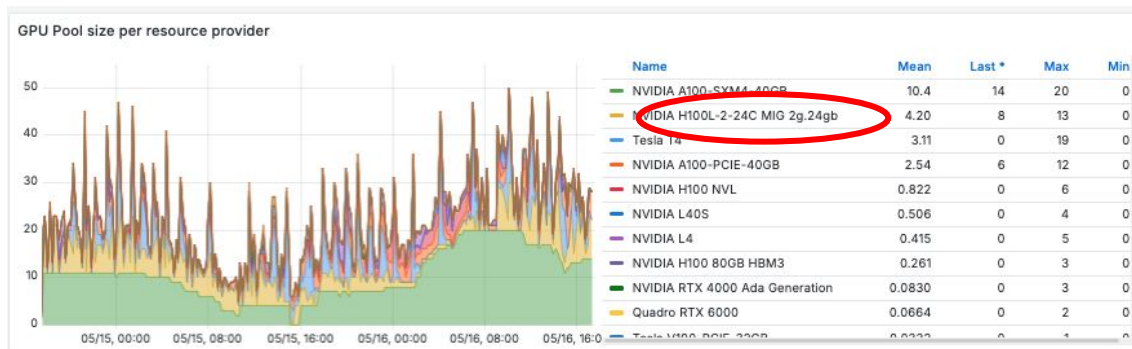
GPU Pool size per resource provider



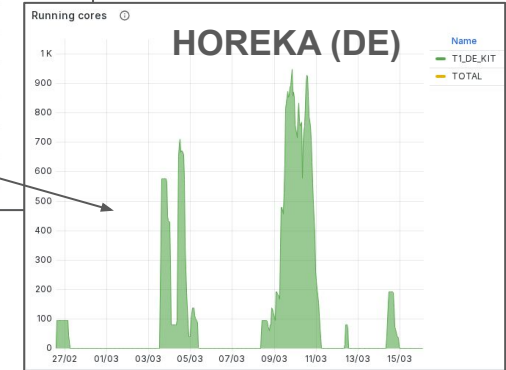
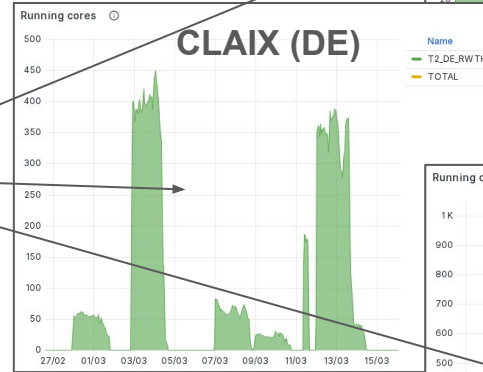
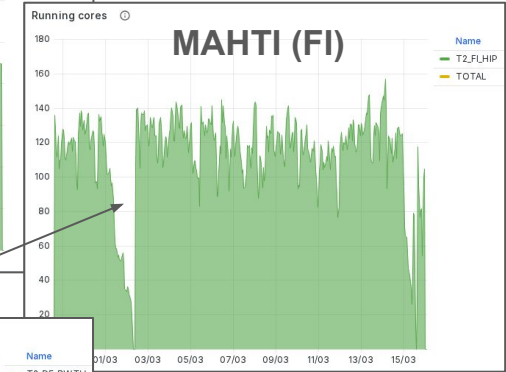
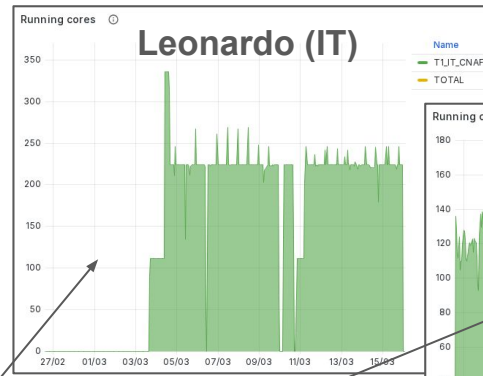
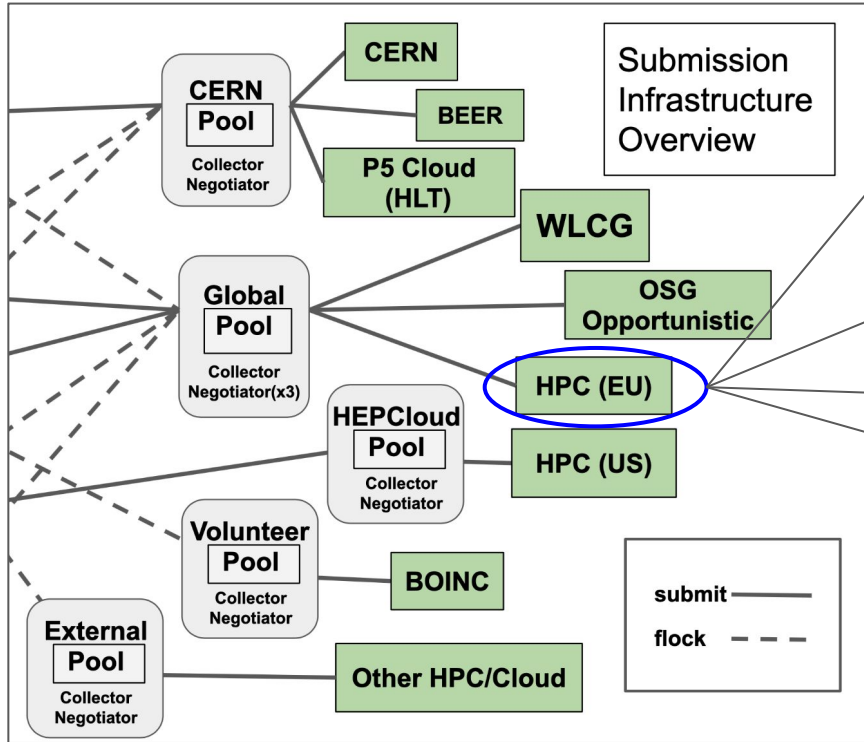
Splitting GPUs

- Payloads often do not require a full GPU (yet(?))
 - Improve utilization by scheduling multiple jobs onto the same GPU
 - KIT working on this, check out Tim's talk [here](#)
- Two approaches for splitting GPU:
 - Multi-Process Service (MPS) and Multi-Instance GPU (MIG)
 - Both approaches are to be investigated by CMS for efficient use of the resources

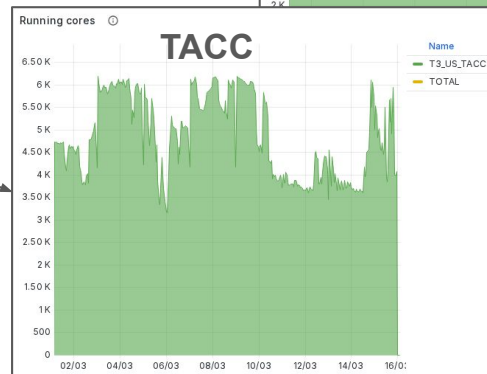
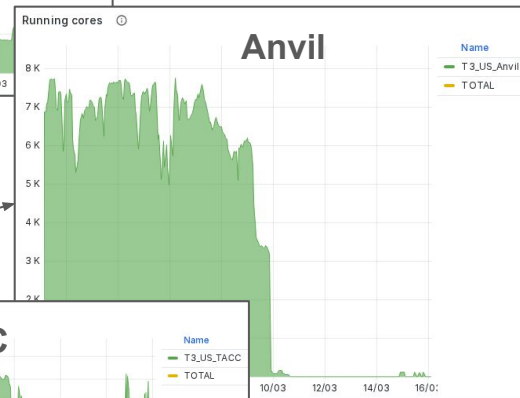
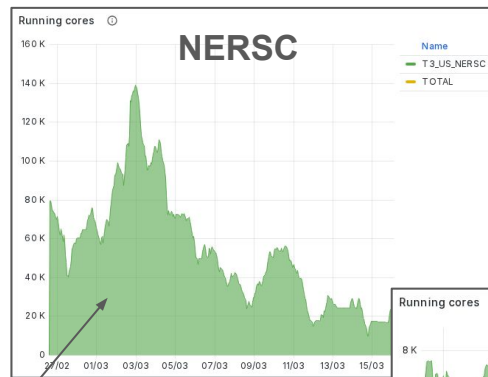
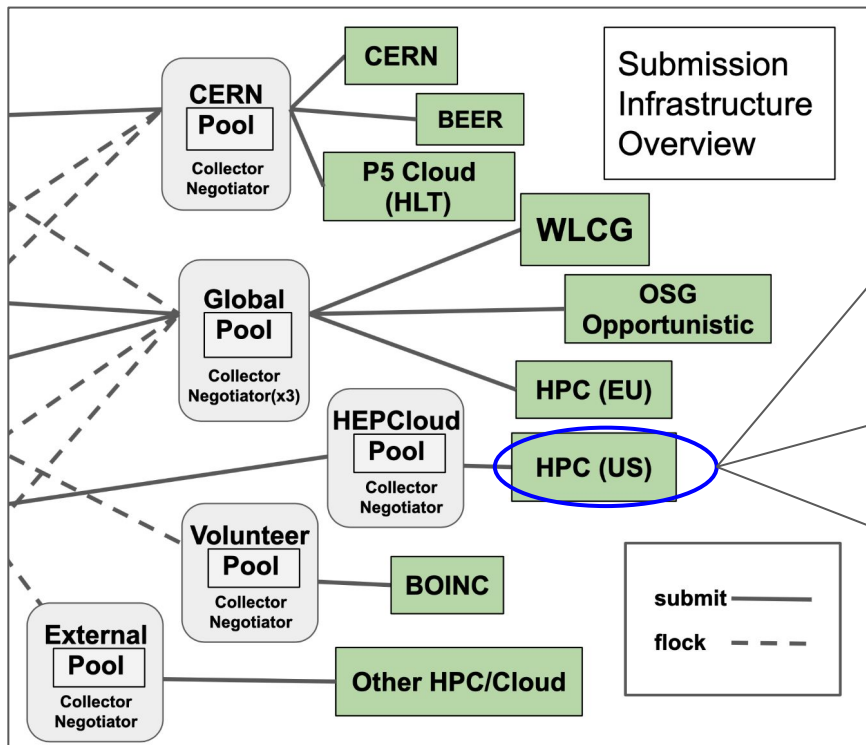
CMS SI working together with different sites (e.g. CERN, KIT) to make this available for CMS



HPC in Europe



HPC in the US



Conclusions

- Support and scheduling of heterogeneous resources like other processor architectures, especially ARM and GPUs is readily available
 - Using HTCondor ClassAds for two-step matchmaking is flexible enough to allow for resource acquisition and job match making
- CMS SI is in a good position for supporting heterogeneous resources as required for the HL-LHC era. Key points to maintain are **stability**, **flexibility**, and **efficiency**
 - HTCondor and GlideinWMS allow to scale out in the HL-LHC era, see also [Antonio's talk later today](#)

Acknowledgements

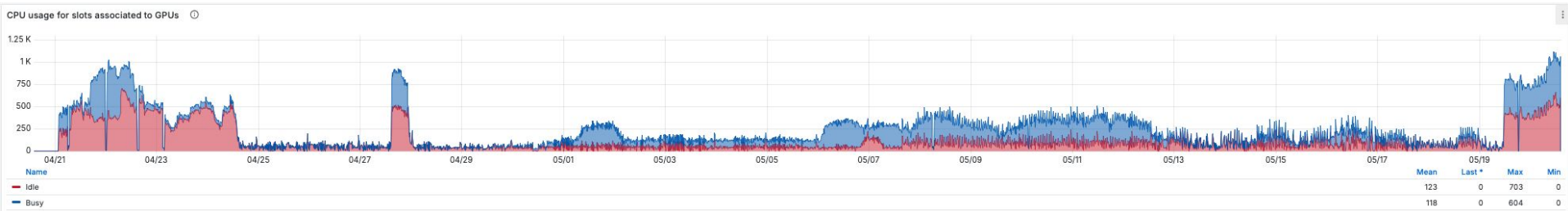
A. Pérez-Calero Yzquierdo's research supported by project PID2022-142604OB-C21, funded by



This work was partially supported by the US National Science Foundation under Grant No. 2121686 and under Grant No. 2030508

Overview of GPUs in the Pools

Usage of CPUs associated to GPU slots



Factory / Entry configuration

```
10-cmst1-uscmst2-all.xml-      <attr name="SINGULARITY_IMAGES_DICT"
const="True" glidein_publish="True" job_publish="True" parameter="True"
publish="True" type="string"
value="default:/cvmfs/unpacked.cern.ch/registry.hub.docker.com/cmssw/el7:aarch
64,rhel8:/cvmfs/unpacked.cern.ch/registry.hub.docker.com/cmssw/el8:aarch64,el7
:/cvmfs/unpacked.cern.ch/registry.hub.docker.com/cmssw/el7:aarch64,el9:/cvmfs/
unpacked.cern.ch/registry.hub.docker.com/cmssw/el9:aarch64"/>
```

Things to add:

- Test with benchmarking team through ITB schedd
 - ARM resources: SI is ready
 - Multi-job GPUs?
 - Usage of MIGs, reference to KIT talk
 - GlobalMatcher in FE
-
- FOCUS MORE ON SCHEDULING PART